

Текст программы (решение варианта А для 32 варианта предметной области)

```

from operator import itemgetter

class Column:
    """Колонка данных"""
    def __init__(self, id, name, data_type, storage_size, table_id):
        self.id = id
        self.name = name
        self.data_type = data_type
        self.storage_size = storage_size # Количественный признак
        self.table_id = table_id # Для связи один-ко-многим

class Table:
    """Таблица данных"""
    def __init__(self, id, name):
        self.id = id
        self.name = name

class ColumnTable:
    """'Колонки таблиц' для реализации связи многие-ко-многим"""
    def __init__(self, table_id, column_id):
        self.table_id = table_id
        self.column_id = column_id

# Таблицы
tables = [
    Table(1, 'Users'),
    Table(2, 'Orders'),
    Table(3, 'Products'),

    Table(11, 'System Users'),
    Table(22, 'System Orders'),
    Table(33, 'Archive'),
]

# Колонки
columns = [
    Column(1, 'UserID', 'int', 100, 1),
    Column(2, 'Username', 'varchar', 250, 1),
    Column(3, 'OrderID', 'int', 150, 2),
    Column(4, 'ProductID', 'int', 120, 2),
    Column(5, 'ProductName', 'varchar', 300, 3),
    Column(6, 'Price', 'decimal', 80, 3),
]

# Колонки таблиц
cols_tabs = [
    ColumnTable(1, 1),
    ColumnTable(1, 2),
    ColumnTable(2, 3),
    ColumnTable(2, 4),
    ColumnTable(3, 5),
    ColumnTable(3, 6),

    ColumnTable(11, 1),
    ColumnTable(11, 2),
    ColumnTable(22, 3),
    ColumnTable(22, 4),
    ColumnTable(33, 5),
]

```

```

def main():
    """Основная функция"""

    # 1. Соединение один-ко-многим
    # (Column.name, Column.storage_size, Table.name)
    one_to_many = [
        (c.name, c.storage_size, t.name)
        for t in tables
        for c in columns
        if c.table_id == t.id
    ]

    # 2. Соединение многие-ко-многим
    # (Table.name, ct.table_id, ct.column_id)
    many_to_many_temp = [
        (t.name, ct.table_id, ct.column_id)
        for t in tables
        for ct in cols_tabs
        if t.id == ct.table_id
    ]

    # (Column.name, Column.storage_size, table_name)
    many_to_many = [
        (c.name, c.storage_size, table_name)
        for table_name, table_id, column_id in many_to_many_temp
        for c in columns
        if c.id == column_id
    ]

    # Вариант А

    # Задание А1
    print('Задание А1')
    res_A1 = sorted(one_to_many, key=itemgetter(2))
    print(res_A1)

    # Задание А2
    print('\nЗадание А2')
    res_A2_unsorted = []
    # Перебираем все таблицы
    for t in tables:
        # Список колонок для этой таблицы
        t_cols = list(filter(lambda i: i[2] == t.name, one_to_many))
        # Если в таблице есть колонки
        if len(t_cols) > 0:
            # Размеры колонок
            t_sizes = [size for _, size, _ in t_cols]
            # Суммарный размер
            t_sizes_sum = sum(t_sizes)
            res_A2_unsorted.append((t.name, t_sizes_sum))

    # Сортировка по суммарному размеру (по убыванию)
    res_A2 = sorted(res_A2_unsorted, key=itemgetter(1), reverse=True)
    print(res_A2)

    # Задание А3
    # Слово «System»
    print('\nЗадание А3')
    res_A3 = {}
    # Перебираем все таблицы
    for t in tables:
        # Ищем 'System' в имени
        if 'System' in t.name:

```

```

# Список колонок для этой таблицы
t_cols = list(filter(lambda i: i[2] == t.name, many_to_many))
# Только имена колонок
t_cols_names = [name for name, _, _ in t_cols]
# Добавляем в словарь
res_A3[t.name] = t_cols_names

print(res_A3)

if __name__ == '__main__':
    main()

```

Результаты выполнения:

Задание A1

[('OrderID', 150, 'Orders'), ('ProductID', 120, 'Orders'), ('ProductName', 300, 'Products'), ('Price', 80, 'Products'), ('UserID', 100, 'Users'), ('Username', 250, 'Users')]

Задание A2

[('Products', 380), ('Users', 350), ('Orders', 270)]

Задание A3

{'System Users': ['UserID', 'Username'], 'System Orders': ['OrderID', 'ProductID']}